

TechBiz Security

Cyber Security Internship Report

Week 6 – Task 18 (SOC)

Brute Force Attack Investigation and Log/Network Traffic Analysis

Submitted By	MANOJ
Role	Cyber Security Intern
Internship Organisation	TechBiz Security
Internship ID	TBS-INT-2026-518
Submitted Date	28 SEPTEMBER 2026

1. Introduction

This report documents the practical work carried out for Week 6 – Task 18 (SOC) of my cyber security internship at TechBiz Security. The task was based on the LetsDefend "Brute Force Attacks" challenge, in which a web server is described as compromised and the analyst is expected to analyze logs, examine network traffic, and identify evidence related to the attack.

From a SOC analyst perspective, a brute-force attack is investigated by looking for repeated authentication attempts and then confirming what the targeted system did in response. Network traffic analysis shows what was sent to the server, which resource was requested, which parameters were submitted, and how the server replied. Authentication log analysis shows which accounts were accepted by the host and from which source addresses. Neither source is sufficient alone, so an investigation uses both. In this lab I inspected an HTTP packet capture in Wireshark and searched an SSH authentication log, then related the two.

2. Objectives

- Investigate evidence of a brute-force attack in the LetsDefend challenge data.
- Analyze captured HTTP traffic using Wireshark.
- Identify repeated login requests and the server responses to them.
- Examine authentication logs for successful login events.
- Identify the targeted server IP address.
- Identify the targeted web resource.
- Correlate network traffic and authentication log evidence.
- Understand how a SOC analyst investigates authentication-related attacks.

3. Lab Environment

The practical work was performed inside the LetsDefend browser-based lab for the Brute Force Attacks challenge. The following components are visible in the screenshots:

Component	Details Observed in the Evidence
Challenge platform	LetsDefend, "Brute Force Attacks" challenge, connected through the Lab Environment Connect option
Working environment	Linux desktop with a terminal session running as root (prompt root@ip-172-31-46-90)
Challenge file	BruteForce.7z at /root/Desktop/ChallengeFile/BruteForce.7z
Archive utility	7-Zip (p7zip Version 16.02) invoked from the terminal
Packet capture	BruteForce.pcap opened in Wireshark
Log file	auth.log, searched from the terminal in the ChallengeFile directory

4. Challenge File and Initial Investigation

The challenge page describes a compromised web server and asks the analyst to analyze logs, dissect network traffic, and uncover clues to identify the attacker and the extent of the damage. It lists the challenge file location as /root/Desktop/ChallengeFile/BruteForce.7z (Figure 1).

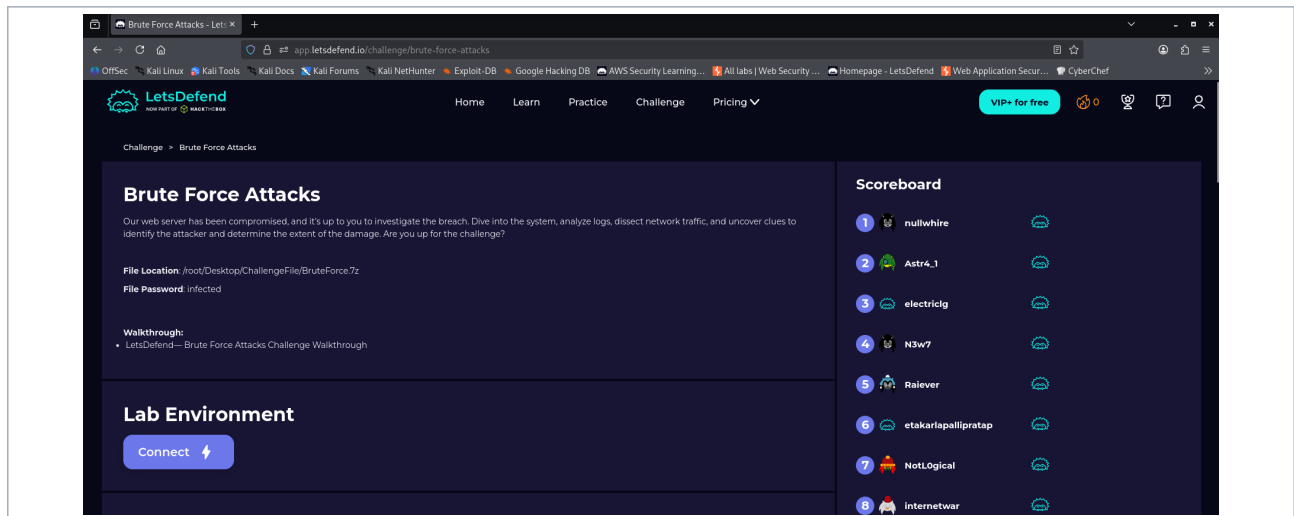


Figure 1: LetsDefend Brute Force Attacks challenge and lab environment.

In the lab terminal, I listed the ChallengeFile directory with `ls -lh /root/Desktop/ChallengeFile/`. The output shows a single file, BruteForce.7z, of 1.9M, owned by the user "analyst" and dated Jul 22 2024. A 7z command was then run against the archive, and the 7-Zip banner (p7zip 16.02) is displayed. This step confirmed the archive was accessible before analysis of its contents (Figure 2).

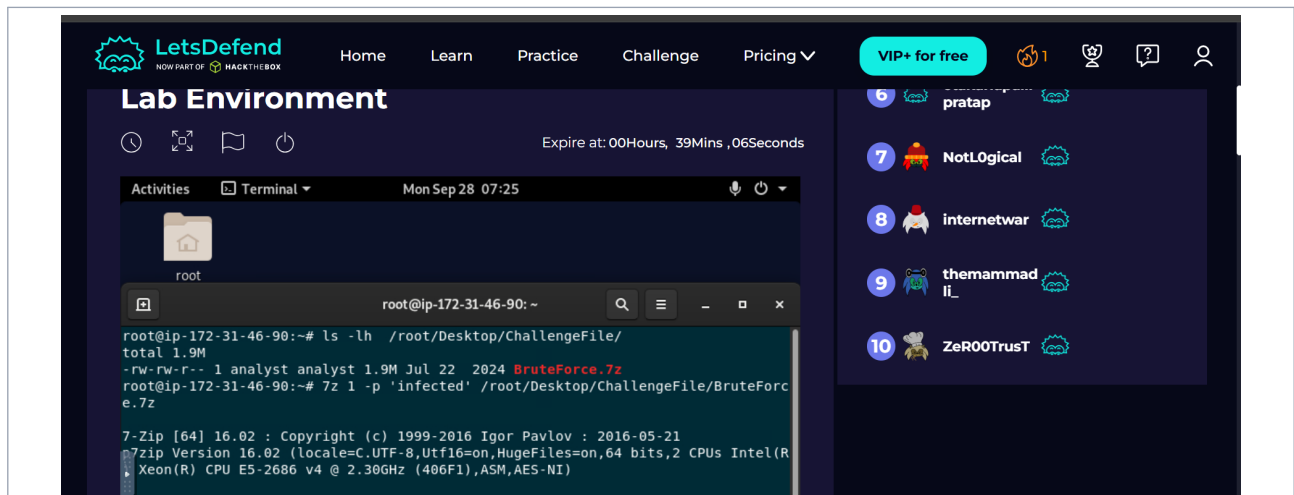


Figure 2: Challenge file BruteForce.7z listed in the lab terminal and accessed with 7-Zip.

5. Network Traffic Analysis Using Wireshark

The packet capture BruteForce.pcap was analyzed in Wireshark. Applying the display filter `http.response.code == 200` isolated HTTP responses with a 200 OK status. Figure 3 shows a long run of these responses, all from 51.116.96.181 (HTTP source port 80) to 192.168.190.137, each of type text/html and 561 to 562 bytes long. The frame numbers increase by 13 between consecutive responses and the timestamps advance by roughly 0.2 seconds, which shows the same exchange being repeated in rapid succession.

Frame 22249 was selected for inspection. The decoded HTTP body shows a page titled "Fake Login Page" with a login heading, and a paragraph styled in green containing the word "Correct" (highlighted

in the screenshot). This response is 561 bytes, one byte smaller than the neighbouring 562-byte responses. I record this as an observation only; the screenshot does not show the bodies of the other responses, so I did not draw a firm conclusion about which attempts succeeded.

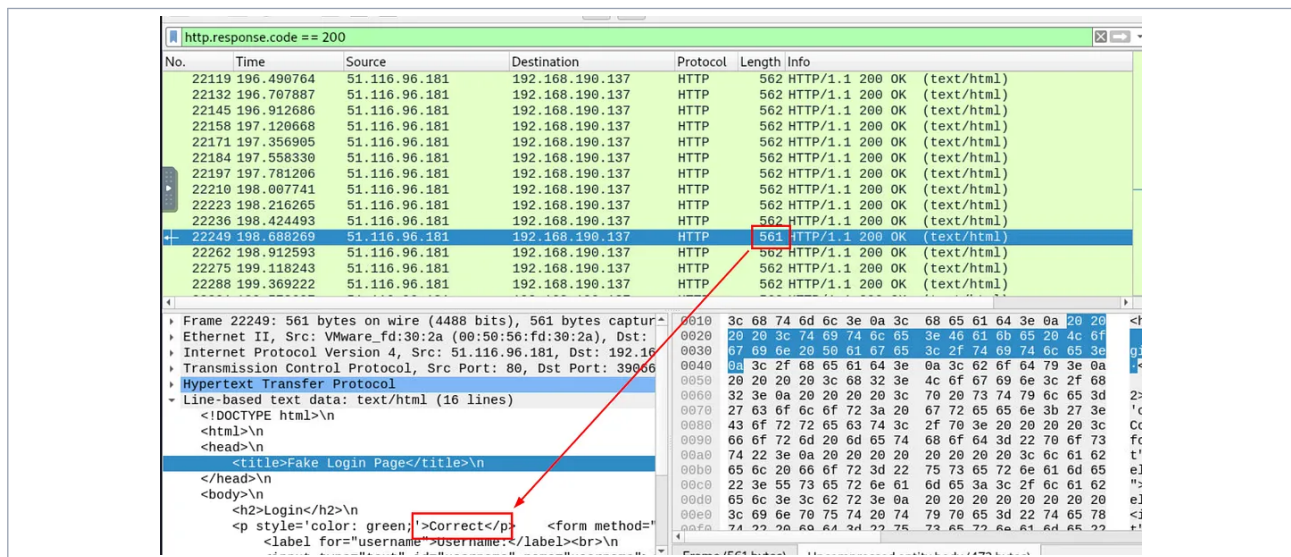


Figure 3: HTTP response analysis showing repeated 200 OK responses and the login page response body (frame 22249).

A separate view of the capture (Figure 4) shows frames 226 to 239. In this section the traffic between 192.168.190.137 and 51.116.96.181 is TCP and RDP on port 3389, including RST, FIN and ACK segments and RDP PDUs (Synchronize, Control: Granted). This shows that the same server address also appears in non-HTTP traffic in the capture. This screenshot on its own does not demonstrate repeated login requests, so I have not used it as evidence of HTTP POST activity; it is included as a record of the additional traffic observed in the packet list.

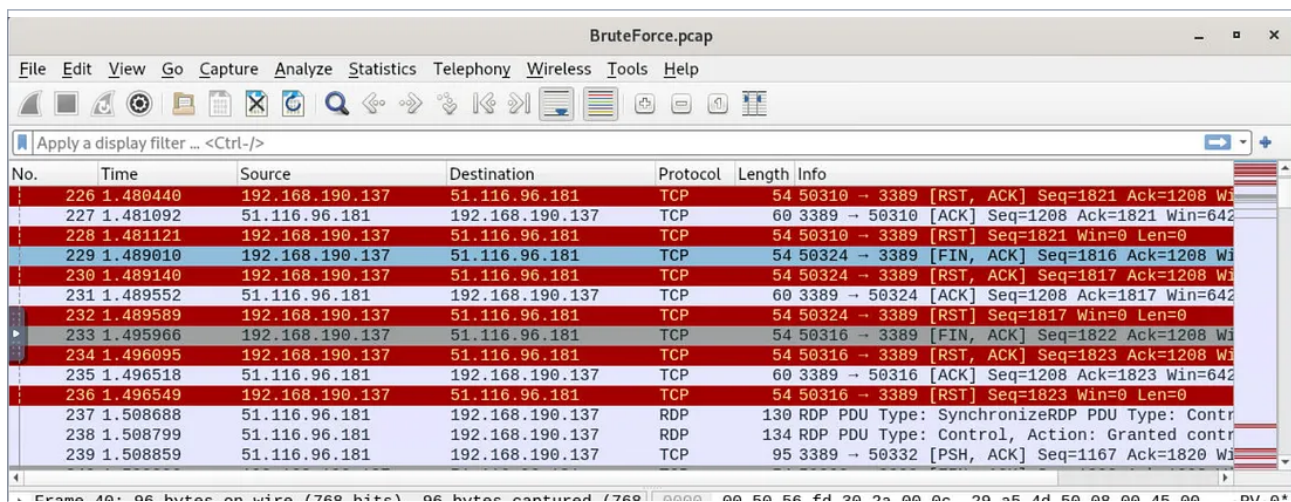


Figure 4: Wireshark packet list of BruteForce.pcap showing TCP and RDP traffic between 192.168.190.137 and 51.116.96.181.

6. HTTP Login Request Analysis

To examine an individual login attempt, I used Wireshark Follow HTTP Stream on TCP stream 352 (Figure 5). The client request contains the following:

```
POST /index.php HTTP/1.1
Host: 51.116.96.181
User-Agent: python-requests/2.31.0
Content-Length: 39
Content-Type: application/x-www-form-urlencoded
```

```
username=web-hacker&password=admin12345
```

The form data is transmitted in clear text over HTTP, so the submitted parameters are readable in the capture. These values are credentials visible in the captured lab traffic only and are not associated with any real-world account. The server answered with HTTP/1.1 200 OK (Date: Sun, 25 Feb 2024 12:39:29 GMT, Server: Apache/2.4.52 (Ubuntu)) and returned the same "Fake Login Page" HTML containing the word "Correct".

Two further details are relevant. The User-Agent is python-requests/2.31.0, which identifies a scripted HTTP client rather than a web browser, and this is consistent with automated login attempts. The request is also directed to /index.php on 51.116.96.181, the same server and resource seen in the repeated responses in Figure 3.

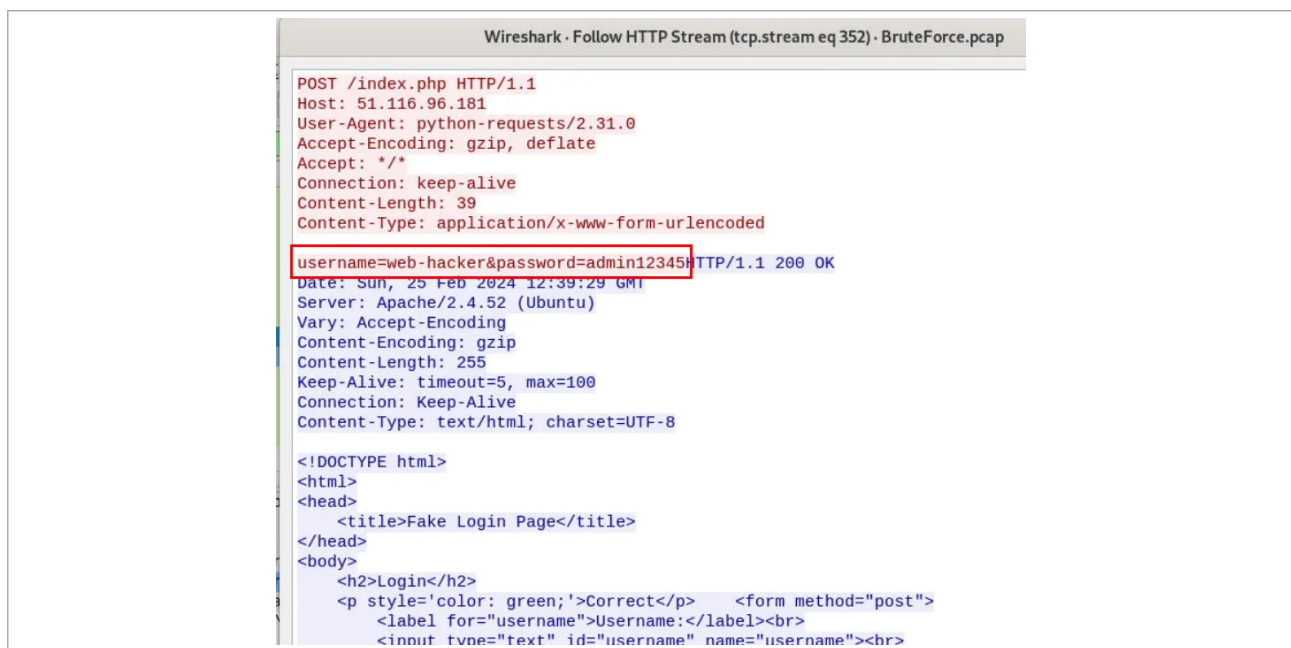


Figure 5: HTTP POST request containing login form data (Follow HTTP Stream, tcp.stream eq 352).

Inspecting POST parameters is valuable to a SOC analyst because it shows the exact username and password combination attempted, the endpoint receiving it, the client software used, and the server reply. Comparing these details across many requests helps distinguish a normal user login from automated credential guessing.

7. Identification of the Target Server

The target server was identified from the packet capture. In every HTTP exchange examined, 51.116.96.181 appears as the server address: it is the Host header and destination of the POST request in Figure 5, and the source of the HTTP 200 OK responses returned to 192.168.190.137 in Figure 3. The LetsDefend question "What is the IP address of the server targeted by the attacker's brute-force attack?" was answered with 51.116.96.181 and marked Correct and Completed.

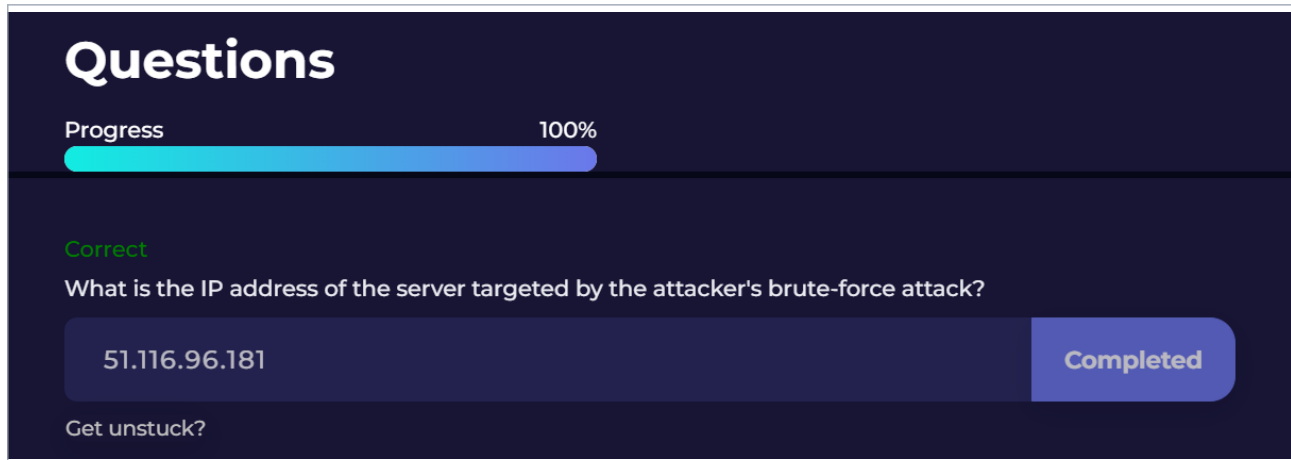


Figure 6: Identification of the server targeted by the brute-force attack.

8. Identification of the Targeted Web Resource

The login requests examined in the capture were directed to /index.php (Figure 5), and the responses returned the login page content (Figure 3). The LetsDefend question "Which directory was targeted by the attacker's brute-force attempt?" was answered with index.php and marked Correct and Completed. The challenge progress indicator shows 100 percent.

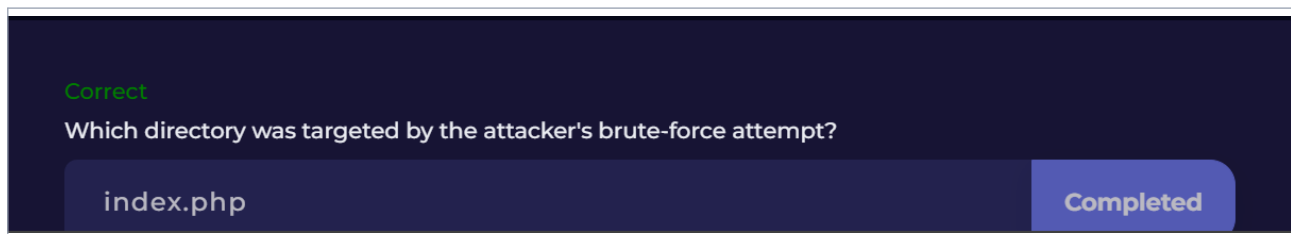


Figure 7: Identification of the targeted web resource (index.php), with challenge progress at 100 percent.

9. Authentication Log Analysis

For host-side evidence, I searched the authentication log from the ChallengeFile directory with the following command:

```
grep -i "accepted" auth.log
```

This command returns every line of auth.log containing the word "accepted" regardless of case, which isolates successful authentication events logged by sshd. The output (Figure 8) shows 27 entries of the form "Accepted password for mmox from [source IP] port [port] ssh2", recorded on the host named chall between Feb 24 10:14:08 and Feb 25 11:43:54.

Source IP Address	Accepted Password Events Visible	Account
196.136.60.15	5	mmox
41.38.160.33	22	mmox

```

root@ip-172-31-10-123:~/Desktop/ChallengeFile# grep -i "accepted" auth.log
Feb 24 10:14:08 chall sshd[1039]: Accepted password for mmox from 196.136.60.15 port 32976 ssh2
Feb 24 10:38:38 chall sshd[2618]: Accepted password for mmox from 196.136.60.15 port 32932 ssh2
Feb 24 10:43:28 chall sshd[2755]: Accepted password for mmox from 196.136.60.15 port 32933 ssh2
Feb 24 10:44:04 chall sshd[2825]: Accepted password for mmox from 196.136.60.15 port 32903 ssh2
Feb 24 13:57:41 chall sshd[5367]: Accepted password for mmox from 41.38.160.33 port 55871 ssh2
Feb 24 13:57:42 chall sshd[5400]: Accepted password for mmox from 41.38.160.33 port 55868 ssh2
Feb 24 14:29:30 chall sshd[6012]: Accepted password for mmox from 41.38.160.33 port 25846 ssh2
Feb 24 14:48:57 chall sshd[6484]: Accepted password for mmox from 41.38.160.33 port 26242 ssh2
Feb 24 14:50:43 chall sshd[6592]: Accepted password for mmox from 41.38.160.33 port 26268 ssh2
Feb 24 14:59:37 chall sshd[6751]: Accepted password for mmox from 41.38.160.33 port 26502 ssh2
Feb 24 17:30:28 chall sshd[8504]: Accepted password for mmox from 41.38.160.33 port 28460 ssh2
Feb 24 18:01:50 chall sshd[9424]: Accepted password for mmox from 196.136.60.15 port 12175 ssh2
Feb 24 20:56:05 chall sshd[3579]: Accepted password for mmox from 41.38.160.33 port 52007 ssh2
Feb 24 21:35:45 chall sshd[5330]: Accepted password for mmox from 41.38.160.33 port 52665 ssh2
Feb 24 21:40:05 chall sshd[5466]: Accepted password for mmox from 41.38.160.33 port 52730 ssh2
Feb 24 22:29:15 chall sshd[7236]: Accepted password for mmox from 41.38.160.33 port 53803 ssh2
Feb 24 23:04:49 chall sshd[7851]: Accepted password for mmox from 41.38.160.33 port 54597 ssh2
Feb 24 23:06:41 chall sshd[7943]: Accepted password for mmox from 41.38.160.33 port 54634 ssh2
Feb 25 10:45:33 chall sshd[14130]: Accepted password for mmox from 41.38.160.33 port 52860 ssh2
Feb 25 10:46:22 chall sshd[14266]: Accepted password for mmox from 41.38.160.33 port 52890 ssh2
Feb 25 11:08:45 chall sshd[14449]: Accepted password for mmox from 41.38.160.33 port 53735 ssh2
Feb 25 11:11:27 chall sshd[15010]: Accepted password for mmox from 41.38.160.33 port 53787 ssh2
Feb 25 11:21:48 chall sshd[15709]: Accepted password for mmox from 41.38.160.33 port 54057 ssh2
Feb 25 11:21:57 chall sshd[15765]: Accepted password for mmox from 41.38.160.33 port 54058 ssh2
Feb 25 11:34:45 chall sshd[16532]: Accepted password for mmox from 41.38.160.33 port 54294 ssh2
Feb 25 11:39:22 chall sshd[18857]: Accepted password for mmox from 41.38.160.33 port 54357 ssh2
Feb 25 11:43:54 chall sshd[981]: Accepted password for mmox from 41.38.160.33 port 54464 ssh2

```

Figure 8: Authentication log analysis showing repeated accepted password events.

The final entry, highlighted in the screenshot, is Feb 25 11:43:54, sshd[981], Accepted password for mmox from 41.38.160.33 port 54464 ssh2. Repeated successful logins for one account from the same source address are worth attention in an incident investigation, particularly at short intervals such as the entries at 13:57:41 and 13:57:42 on Feb 24. However, an Accepted password event alone does not show how the password was obtained. I therefore treat these events as leads to be correlated with other evidence, and I do not state that each one resulted from the brute-force attack. The screenshot also shows only accepted events; failed attempts were not searched in the evidence provided.

10. Correlation of Evidence

The table below separates what was directly observed from the interpretation I drew from it.

Evidence Source	Observed	Interpretation
Challenge page (Fig. 1)	Description of a compromised web server requiring log and traffic analysis	The exercise is framed as a web server attack investigation
Wireshark 200 OK filter (Fig. 3)	Many HTTP 200 OK responses from 51.116.96.181 about 0.2 s apart; frame 22249 body contains "Correct"	The login endpoint was being called repeatedly in a short period, consistent with automated attempts
Follow HTTP Stream (Fig. 5)	POST /index.php with username=web-hacker&password=admin12345; User-Agent python-requests/2.31.0	Credentials were submitted in clear text by a scripted client to the login resource
Question answers (Figs. 6, 7)	Target IP 51.116.96.181; targeted resource index.php	Confirms the server and resource identified from the traffic

Evidence Source	Observed	Interpretation
auth.log search (Fig. 8)	27 Accepted password events for mmox from 196.136.60.15 and 41.38.160.33 over Feb 24 - Feb 25	Host-side successful SSH logins exist for one account from two sources; their link to the HTTP activity is not established by these screenshots

A SOC analyst correlates these sources by aligning time, address and account. Network traffic gives the sequence of attempts and the target, while the host log gives the outcome as recorded by the system. In this lab, the HTTP capture and the SSH log are consistent with an authentication attack but are different services, so linking them further would need matching timestamps and clock references. The pcap response header shows 25 Feb 2024 12:39:29 GMT, while auth.log entries carry no year or time zone, so I did not attempt a time-based match. The source addresses 196.136.60.15 and 41.38.160.33 in the log do not appear in the Wireshark screenshots provided, which show 192.168.190.137 as the client.

11. Findings

No.	Finding	Evidence
1	The investigated challenge involved brute-force attack activity against a web server.	Figure 1, Figure 6
2	The targeted server IP address was 51.116.96.181.	Figures 3, 5, 6
3	The targeted web resource was /index.php.	Figures 5, 7
4	Numerous HTTP 200 OK responses were returned by the server in rapid succession.	Figure 3
5	An HTTP POST request to /index.php was inspected and contained login form data.	Figure 5
6	Credentials (username=web-hacker, password=admin12345) were visible in clear text in the captured lab traffic.	Figure 5
7	The request used the User-Agent python-requests/2.31.0, indicating a scripted client.	Figure 5
8	The response body of frame 22249 contained the word "Correct" on a "Fake Login Page".	Figure 3
9	Additional TCP/RDP (port 3389) traffic between 192.168.190.137 and 51.116.96.181 was present in the capture.	Figure 4
10	auth.log contained 27 visible Accepted password events for the user mmox.	Figure 8
11	Source IP addresses in the log were 196.136.60.15 (5 events) and 41.38.160.33 (22 events).	Figure 8

12. Security Impact

Password-based authentication attacks succeed when weak, guessable or reused passwords are accepted and when a service allows unlimited attempts. The traffic examined here shows a login form served over plain HTTP, so submitted usernames and passwords can be read by anyone able to capture the traffic. Repeated login attempts in a short period, especially from a scripted client, place load on the service and increase the chance that a weak password is guessed.

If such attempts succeed, an attacker could gain unauthorized access to the application or host. The screenshots I analyzed do not by themselves prove that an account was compromised through the brute-force activity; they show attempts, responses, and separate successful SSH logins that require further correlation. This is why monitoring authentication logs and correlating network and host evidence is important: it allows an analyst to determine whether attempts led to real access and what the extent of any damage may be.

13. Mitigation and Recommendations

- Enforce strong and unique passwords and block commonly guessed values.
- Implement account lockout or rate limiting on login endpoints.
- Use multi-factor authentication (MFA) for important accounts, including SSH access.
- Monitor repeated authentication failures and any successes that follow them.
- Centralize authentication logs so they can be searched and retained.
- Alert on suspicious or unexpected source IP addresses.
- Use HTTPS instead of HTTP for all login forms so credentials are not exposed in transit.
- Protect login endpoints against automated requests, for example with CAPTCHA or blocking of scripted clients.
- Implement secure session management after successful authentication.
- Regularly review authentication logs for unusual patterns.
- Correlate network-based and host-based security events during investigations.

14. SOC Analyst Learning Outcomes

Through this exercise I gained practical experience in the following areas:

- Recognizing brute-force indicators such as rapid repeated requests to a login resource and a scripted User-Agent.
- Using Wireshark display filters and Follow HTTP Stream for packet analysis.
- Inspecting HTTP requests and responses, including headers, status codes and response bodies.
- Identifying authentication parameters in HTTP form data.
- Searching authentication logs with grep to isolate successful logins.
- Correlating host and network evidence while recognizing the limits of what each source proves.
- Identifying the attack target from traffic, both the server address and the web resource.
- Understanding that evidence must be separated from interpretation in SOC reporting.

15. Conclusion

This practical exercise demonstrated how a SOC analyst can investigate brute-force activity by combining packet capture analysis, HTTP request inspection, authentication log analysis, and challenge evidence. I identified 51.116.96.181 as the targeted server and /index.php as the targeted web resource, observed a clear-text login POST from a scripted client, and found repeated accepted SSH password events for the user mmox in auth.log. The evidence supports the conclusion that automated authentication attempts occurred against the login resource. Confirming the full impact would require additional correlation between the network and host data.

16. Evidence / Screenshot Index

Figure	Screenshot Description
Figure 1	01_LetsDefend_Brute_Force_Challenge.png - LetsDefend Brute Force Attacks challenge page with lab environment connect option
Figure 2	02_Lab_Environment_Challenge_File.png - Lab terminal showing BruteForce.7z listing and 7-Zip access
Figure 3	03_Wireshark_HTTP_Response_Analysis.png - Wireshark http.response.code == 200 filter and login page response body
Figure 4	04_Wireshark_TCP_RDP_Packet_List.png - Wireshark packet list showing TCP and RDP traffic with 51.116.96.181
Figure 5	05_Wireshark_HTTP_Login_Request.png - Follow HTTP Stream showing POST /index.php and login form data
Figure 6	06_LetsDefend_Target_Server_IP.png - Completed question: target server IP 51.116.96.181
Figure 7	07_LetsDefend_Targeted_Web_Resource.png - Completed question: targeted resource index.php (Figures 6 and 7 are taken from one screenshot)
Figure 8	08_Authentication_Log_Accepted_Passwords.png - grep -i "accepted" auth.log output

17. References

- LetsDefend, "Brute Force Attacks" challenge, LetsDefend platform (challenge page shown in Figure 1).
- Wireshark User's Guide, Wireshark Foundation (documentation for display filters and Follow HTTP Stream).
- Linux sshd and authentication logging documentation (manual pages for sshd and grep, and the system authentication log auth.log).